

MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 5 — APIs and Network Communication

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology

Last week: What must the system remember?

- Persistent entities
- Tables, keys, and relationships
- State transitions
- Migrations
- Database as system truth

From Persistent State to Networked Communication

This week: How do components communicate about that state?

User / Client

↓ HTTP

API → Database → Worker → AI-enabled service

↑

↓

└── status / result ──┘

The client should not need direct access to the database, worker, or AI service.



Today's Focus

Today we will focus on:

- REST-style API design
- Request and response contracts
- Validation and error handling
- Internal and external service boundaries
- Network failure behavior
- Idempotency and retry safety
- Authentication and authorization fundamentals
- Interface discipline for the Week 7 vertical slice

Key connection:

API design connects user workflows, persistent data, workers, AI integration, and testing.



Learning Goals

By the end of today, we should be able to:

- Define the main interfaces in our system
- Distinguish between internal and external interfaces
- Design request and response contracts
- Reason about validation and error handling
- Identify clear service boundaries
- Explain idempotency and safe retries
- Apply authentication and authorization at an appropriate scope
- Prepare a coherent vertical-slice interface plan

Key idea:

An endpoint is not just a URL. It is a contract.

Interfaces Are Where Assumptions Become Concrete

An interface answers:

- Who can call this?
- What input is accepted?
- What validation occurs?
- What records are read or changed?
- What response is returned?
- What can fail?
- How should the caller retry?
- Who owns this behavior?

- **Endpoint $\neq$ URL only**

- **Endpoint = path + method + input + output**

- **+ validation + errors + side effects**

- **+ security + retry behavior**

Start With One Primary Workflow



Model Communication From One Real Workflow

- Example:
- User submits a case.
- Client sends POST /cases.
- API validates the request.
- API creates a case and processing job.
- API returns initial state: queued.
- Worker performs AI-enabled processing.
- Worker persists result or failure.
- Client retrieves current state with GET /cases/{id}.

Ask at every boundary:

- What data crosses this boundary?
- Who owns the next state change?
- What happens if the next component is unavailable?

Define the Contract Before Writing the Endpoint

A Useful API Contract Includes

Contract element	Questions to answer
Caller	Who uses this interface?
Method and path	What action or resource does it represent?
Request	Which fields are required? Which are optional?
Validation	What makes the request invalid?
Success response	Which status code and fields are returned?
Error response	Which failures can the caller understand?
Side effects	Which records or states change?
Authentication	Who is allowed to call it?
Retry behavior	Is retry safe? Is it idempotent?

Rule: Design the contract before implementation details spread across the codebase.

Do Not Expose Database Structure Automatically

Client request schema



API route / service logic



SQLAlchemy models and database tables



Response schema for client

Database Models Are Not API Models

The API should expose what the caller needs

- Database-only fields may include:
- Internal foreign keys
- Retry counts
- Worker error details
- Internal timestamps
- AI implementation metadata
- Audit or administrative fields

Key rule:

A request schema, database model, and response schema may overlap—but they have different responsibilities.

REST-Style Design: Resources First

REST-Style Endpoint Design

Use clear resource names:

Purpose	Example
Create a case	POST /cases
Retrieve a case	GET /cases/{case_id}
List or filter cases	GET /cases?status=queued
Update permitted case fields	PATCH /cases/{case_id}
Retry a failed case	POST /cases/{case_id}/retry

Prefer:

- Nouns for resources: /cases, /reviews, /results
- HTTP methods for intent: GET, POST, PATCH, DELETE
- Explicit action endpoints when the domain genuinely has an action

Avoid vague designs:

```
# Not ideal design
POST /doEverything
POST /createCaseNow
GET /getAllData
```

Example Contract: Create a Case

Example: POST /cases

Request

```
{
  "title": "Production outage alert",
  "description": "Several users report the service is down."
}
```

Success response — 201 Created

```
{
  "id": "case_123",
  "title": "Production outage alert",
  "status": "queued",
  "job_id": "job_456",
  "created_at": "2026-10-10T09:30:00Z"
}
```

Contract decisions

- API validates input before persistence.
- API creates the case and job together.
- API returns initial system state.
- API does **not** wait for AI processing to finish.

Return Current State Clearly

Example: GET /cases/{case_id}

Possible states

```
queued → processing → completed  
                ↘ failed
```

Example response — 200 OK

```
{  
  "id": "case_123",  
  "title": "Production outage alert",  
  "status": "completed",  
  "analysis": {  
    "label": "high_priority",  
    "confidence": 0.91,  
    "model_version": "v1"  
  },  
  "updated_at": "2026-10-10T09:32:14Z"  
}
```

Design questions

- What should be visible while processing is incomplete?
- Is an AI result absent, null, or represented differently?
- Which fields are user-facing versus internal?

Validation Happens at Multiple Layers

Validation Is More Than Checking JSON Shape

Layer	Example responsibility
Request schema	Required fields, types, lengths, formats
Business rules	Title is meaningful; submission is allowed
State-transition rules	Only failed work may be retried
Authorization rules	Caller may access this case
Database constraints	Foreign keys, required fields, uniqueness

Client input



Pydantic / request schema



Route and service rules



Database constraints

Rule:

Validate early but keep database constraints as a final safeguard.

Error Handling Is Part of the Interface

Error Responses Should Be Predictable

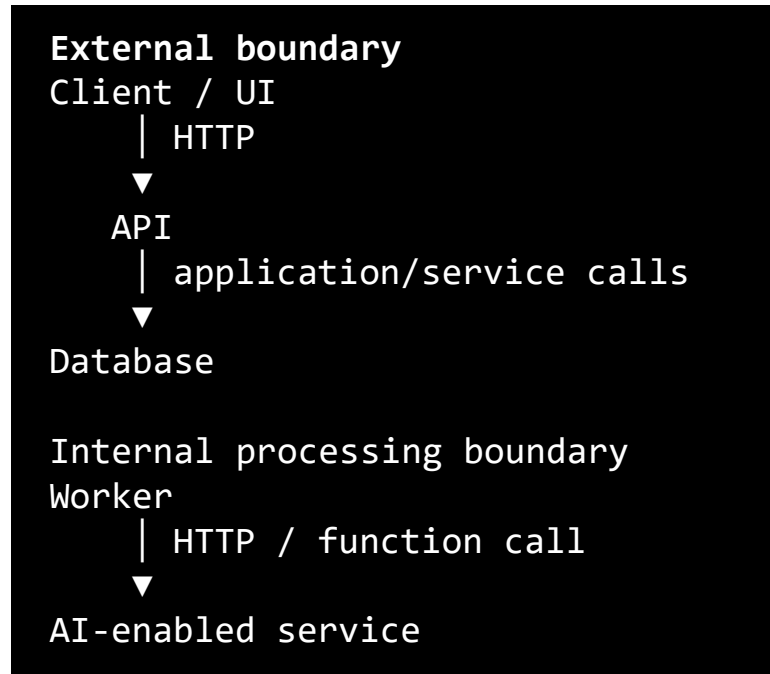
Situation	Typical response
Missing or invalid request fields	422 Unprocessable Content
Record does not exist	404 Not Found
Invalid state transition	409 Conflict
Authentication missing or invalid	401 Unauthorized
Authenticated but not permitted	403 Forbidden
Unexpected server failure	500 Internal Server Error

```
{
  "error": {
    "code": "INVALID_CASE_STATE",
    "message": "Only failed cases may be retried.",
    "request_id": "req_789"
  }
}
```

Do not expose: stack traces, secrets, database credentials, or internal implementation details.

Service Boundaries Clarify Responsibility

Internal and External Interfaces



Interface	Primary concern
Client → API	Usability, validation, authorization, stable response
API → database	Consistent state and transactions
Worker → database	Safe state updates and retries
Worker → AI service	Timeout, input/output contract, failure handling

A boundary is about responsibility—not about creating unnecessary microservices

Network Communication Can Fail Normally

Treat Network Failure as Normal

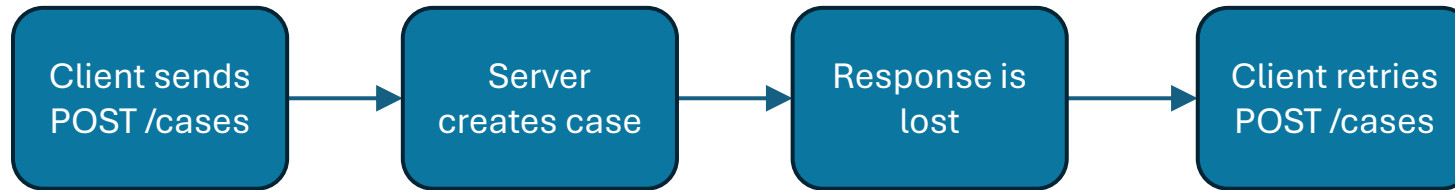
Failure	Example response
AI service unavailable	Worker records failure or schedules retry
AI call exceeds timeout	Stop waiting; mark attempt as failed
Response is malformed	Reject result; preserve diagnostic evidence
Client loses connection after POST	Client may retry request
Worker crashes mid-processing	Job remains recoverable through persisted state

Design principles

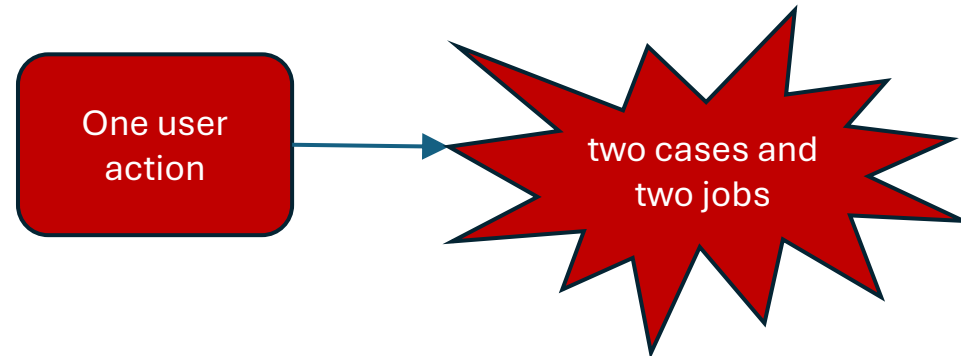
- Use finite timeouts.
- Do not wait indefinitely.
- Record meaningful failure state.
- Retry only when safe and bounded.
- Keep long-running AI work outside the user request cycle.

Idempotency: Safe Handling of Repeated Requests

What If the Same Request Arrives Twice?



Without protection:



Idempotency approach

- Client sends an Idempotency-Key
- Server associates key with caller, endpoint, and request
- Same valid replay returns the original result
- Same key with different request content is rejected

Use idempotency where duplicate creation or duplicate actions would matter.



Authentication and Authorization at an Appropriate Scope

Authentication (who are you?)

Authorization (what are you allowed to do?)

Examples:

- A user may create a case.
- A user may retrieve only their own case.
- A reviewer may approve or reject a result.
- A worker may update processing state.
- An internal AI service should not be publicly exposed.

Course-scope guidance

- If the system is local/demo-only, state that clearly.
- If users and roles matter, implement a minimal testable rule.
- Do not claim production-grade security without evidence.
- Never log passwords, tokens, or sensitive secrets.



Interface Discipline and Contract Evolution

Interfaces Will Change—Change Them Deliberately

Safer changes:

- Add optional response fields
- Add a new endpoint
- Add a new documented status when clients can handle it
- Add filters or pagination without changing existing meaning

Riskier changes:

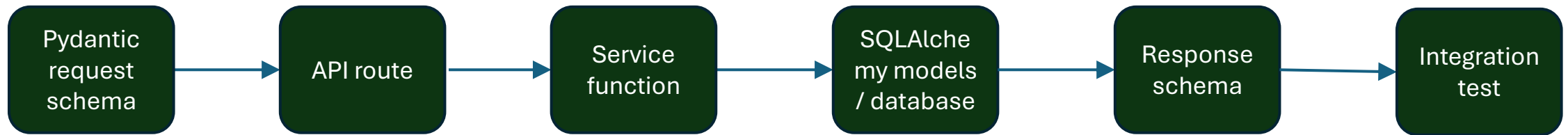
- Rename or remove existing fields
- Change a field's meaning
- Change status behavior without documentation
- Return a different response shape unexpectedly
- Expose internal fields accidentally

Useful practices

- Maintain endpoint examples
- Use FastAPI/OpenAPI documentation
- Review API changes with the team
- Update tests when contracts change
- Version only when a breaking change genuinely requires it

Read and Test Interfaces as a Connected System

Trace One Interface Through the Repository



Test questions

- Does valid input create the intended records?
- Does invalid input return a predictable error?
- Does a missing record return 404?
- Does a forbidden state transition return the expected result?
- Does the response match the documented contract?
- Does the worker/AI path update the API-visible state?

Team studio output

Create an endpoint table for your primary workflow.

Clear Interfaces Make Integration Possible

The Interface Design Habit

Clear workflow

- + explicit request/response contracts
 - + controlled validation and errors
 - + clear service boundaries
 - + safe retry behavior
 - + realistic security scope
- = buildable multi-component system



In-Class Studio

Activity: API and Service Boundary Design

Team task — 15 minutes

For your primary workflow, create the following table:

Method / Path	Caller	Request	Success response	Errors	State/data changes	Auth / retry notes
POST /...						
GET /.../{id}						
PATCH or action endpoint						

Then answer:

1. Which endpoint begins the workflow?
2. Which fields are accepted from the user?
3. Which fields are generated by the system?
4. Which status changes may the API make?
5. Which status changes belong only to the worker?
6. What is the visible failure response?
7. Which endpoint will be demonstrated in Week 7?
8. What will be cut if the interface design becomes too broad?